图麦笔记

> **让有价值的信息，不止于收藏。**

微信小程序参赛说明文档

图麦笔记面向“内容已经收藏，却难以整理和再次使用”的日常问题，把公开链接、手机截图与手写内容汇入同一个笔记空间。链接与截图通过正文提取、文字识别和 AI 提炼，形成标题、摘要、核心要点与标签；用户可以继续编辑、分类、置顶、搜索，并生成带小程序码的笔记卡片图。

作品不是另一个等待用户从空白页开始写作的编辑器，而是连接“发现内容—整理信息—再次使用”的轻量工具。创新重点是记录起点、统一信息结构和卡片回流体验的组合设计，不是自研基础模型。

项目	说明
作品名称 / 形态	图麦笔记 / 微信原生小程序
开发主体	个人开发者；产品经理背景，独立承担产品与工程整合
核心用户	学习资料积累者、职场信息整理者、内容创作者及兴趣记录者
核心技术	原生小程序 + Python 服务 + 开源 OCR + 微信云开发 AI
版本口径	本地代码标识 v1.1.15；不等同于已发布版本证明
文档核对日期	2026 年 9 月 23 日

阅读导航

第 2—3 页：实际问题、应用场景与功能闭环。第 4 页：创新设计及其边界。第 5—7 页：技术开发方案、平台能力与数据处理。第 8—9 页：交互设计和界面展示。第 10 页：团队情况与验证方法。第 11—12 页：第三方成果引用与技术证据。

本说明依据作者原稿和当前本地源码整理。历史截图与历史测量单独注明，不将规划功能、代码存在或旧测试数字当作当前线上效果。参赛者实名与报名主体以赛事报名表为准。

问题不是存不下，而是找不回、用不上

做产品经理的这些年，我习惯把有价值的内容先保存下来：公众号文章留在微信收藏，课程重点留在截图相册，行业资料散落在群聊链接里。真正需要引用某条信息时，却经常记得“看过”，记不得“放在哪”；即使找到了，还要重新阅读和摘录。

这段个人经历是作品的需求起点，不是普遍用户的统计结论。它暴露出的断点很具体：收藏保存了入口，却没有自动完成理解与整理；图片保住了画面，但画面里的文字难以直接检索；信息来源越多，再次使用时切换与寻找的成本越高。

> **把“存下来”与“整理好”之间重复发生的摘录、概括和组织工作，尽可能交给工具；把最终判断留给用户。**

让功能落在真实任务上

使用场景	已有材料与实际困难	在图麦笔记中的完成方式
学习与复习	课程截图、公开教程分散保存，复习时需逐张翻找。	导入截图提取文字，整理要点；再用分类与关键词查回原文。
工作与研究	行业文章看过即忘，开会前还要重复浏览和摘录。	粘贴可访问的文章链接，保留来源，提炼摘要并补充自己的判断。
内容创作	灵感来源不同，参考素材与个人想法难以放在一起。	链接和截图形成笔记；手写补充观点，用标签、置顶建立当前关注集合。
旅行与日常兴趣	攻略有用信息埋在长文里，出门时不方便反复查找。	整理路线要点并生成卡片图，保存相册或打印；联网扫码查看有效分享。

价值如何判断

本阶段目标是缩短“发现内容到留下可用记录”的步骤，并让用户之后能找到它。有效性应通过任务完成率、整理耗时、搜索找回率和再次使用情况验证；目前没有足够的用户研究数据，因此不宣称提升了某个百分比，也不将作者个人习惯当作行业调研结论。

场景说明是产品使用方式，不代表已取得学校、企业或机构合作，也不涉及未经证实的用户规模。

三个入口，一套可继续整理的笔记

功能	当前实现	需要说明的边界
链接转笔记	提交公开 HTTP/HTTPS 链接，抓正文后异步提炼，保留来源链接与原文。	登录页、付费墙、反爬或无有效正文时可能失败；不是任意网站通用解析器。
截图 / 拍照转笔记	从相册或相机导入，一批最多 9 张；自建服务 OCR 后共用提炼链路。	主要支持 JPG、PNG；识别受清晰度、版式、文字大小影响，不承诺全格式无误识别。
手动撰写与编辑	手写直接保存；支持修改标题、摘要、要点、标签和分类。	手写入口不自动调用 AI；AI 输出需由用户复核。
分类、置顶与搜索	自建分类，调整顺序；置顶笔记；关键词匹配标题、摘要、正文及原文。	当前是文本匹配，不是语义问答，也不承诺标签搜索。
笔记卡片与分享	端侧 Canvas 按内容排版导出图片；小程序码进入公开摘要页。	新生成分享有效期 7 天；公开页不等于完整私有笔记，见第 9 页。
账号与外观	微信登录、用户数据隔离、六套壁纸、笔记额度与邀请奖励、自助注销。	本地代码有基础 100 篇与邀请奖励规则；线上以实际部署和页面提示为准。

一条典型任务

选择链接或截图 → 提交采集 → 查看处理状态 → 打开生成的笔记 → 对照原文修正摘要和标签 → 归入分类或置顶 → 日后搜索查回，或生成卡片图使用。手写入口直接进入保存与整理步骤。

失败不能伪装成成功

抓不到正文、OCR 没识别到文字或保存条件不满足时，应提示任务失败；只有文字已经取得、但 AI 提炼不可用时，后端才保留原文并标记降级。当前前端未为所有降级情况提供专门提示，说明文档不把它描述成“摘要始终生成成功”。

证据：miniprogram/pages/create/create.js；backend/app/api/routes/notes.py、categories.py、shares.py；backend/app/services/quota.py；backend/app/tasks/ingest_tasks.py。

让成熟技术形成新的使用闭环

把记录起点，从空白页前移到已经看到的内容

碎片化收集时，用户手里通常已有一段材料，却还没有准备好写一篇笔记。图麦笔记先接住那条链接或那张截图，再辅助完成概括和组织，让记录可以从一次粘贴、一次选图开始。手写入口仍然保留，用于表达自己的判断，而不是强迫所有输入都经过 AI。

不同来源汇入同一结构，同时保留来源

链接抓取与截图 OCR 是两种技术，但输出共用标题、摘要、要点和标签。用户无需为两种来源建立两套整理习惯。统一结构也不意味着抹去出处：链接保留来源地址，识别或抓取文字保留为原文，便于对照、编辑和发现 AI 遗漏。

把使用出口从列表延伸到一张卡片

笔记不必永远留在应用列表中。端侧按照内容测量和绘制卡片，可以存入相册、用于授权范围内的分享，或打印到纸质手账。卡片上的小程序码把纸面和数字内容连接起来，但在线回流受分享有效期与笔记删除状态约束；图片本身可离线查看，扫码并非永久可用。

让低成本、可维护与可信边界一起进入设计

开源 OCR 承担文字识别，平台模型承担语义提炼，确定性的保存与检索由业务服务负责。模型不是唯一成功路径：AI 暂不可用时仍可保留已取得的文字。与此同时，产品不应把限期赠送额度写成永久免费，也不应把“自建 OCR”写成“所有数据不经过第三方”。

原创贡献	不作原创声明的部分
需求定义、三入口组织、统一笔记结构、卡片与回流机制	通用大模型、OCR 模型及其训练方法
异步任务整合、错误与降级策略、资源限制、权限和分享规则	FastAPI、RapidOCR、ONNX Runtime、微信平台 API 等
界面信息层级、设计令牌、Canvas 内容自适应排版与测试	第三方产品名称、通用设计范式及外部内容版权

上述创新是本作品的应用与工程设计，不声称行业首创、专利授权，或竞争产品都不具备相近能力。

确定性业务打底，AI 作为增强层

> **微信原生小程序 → 自建 API → 异步任务与内容处理 → 统一笔记存储 → 搜索、编辑与卡片回流**

层级	技术选型	责任边界
客户端	JavaScript / WXML / WXSS；微信原生 API；Canvas	登录、采集入口、状态展示、笔记整理、卡片渲染与相册导出。
业务 API	Python；FastAPI / Uvicorn；Pydantic；JWT	身份验证、输入约束、额度、分类、笔记和公开分享接口。
任务层	Redis + RQ worker	承接耗时任务；抓取、OCR 与提炼在后台执行；回传任务状态。
内容获取	HTTPX；trafilatura；BeautifulSoup / lxml	访问用户提供的公开网页并抽取正文，不依赖第三方解析服务。
文字识别	RapidOCR；ONNX Runtime；OpenCV / Pillow	在自建服务器 CPU 上执行 OCR 和图像预处理。
语义提炼	Node.js 云函数；wx-server-sdk；cloud.ai()	调用微信云开发模型，接收文字并返回结构化内容。
数据与部署	SQLAlchemy / Alembic；Linux / systemd；反向代理	笔记与用户数据管理、迁移和服务运行；默认数据库配置为 SQLite。

两条采集链路在提炼阶段合流

链接链路：提交地址 → API 校验并入队 → worker 抓取正文 → 云函数提炼 → 笔记入库。截图链路：上传图片 → 批次暂存与入队 → worker 识别文字 → 云函数提炼 → 笔记入库。手写链路不走抓取、OCR 或模型，直接保存用户内容。

关键的不变量

私有笔记始终归属于登录用户；后台任务落库前重新确认用户与保存条件；公开分享只输出明确允许的字段；模型异常不应抹掉已经成功获取的文字。代码支持 PostgreSQL 驱动不等于当前线上已经采用 PostgreSQL，本文不据依赖列表推断部署状态。

证据：backend/requirements.txt；backend/app/core/auth.py；backend/app/tasks/ingest_tasks.py；backend/app/services/queue.py；backend/deploy.sh。

微信云开发 AI，让个人开发的试错更轻

为什么选择微信云开发 AI

个人开发者需要的不只是一个模型接口，还包括接入、鉴权、部署和持续维护的可承受性。图麦笔记利用微信云开发提供的模型入口，把基础推理能力交给平台，开发者集中精力处理输入结构、业务规则和用户体验。

当前实现由自建后端通过带服务端鉴权的 HTTP 请求调用云函数，云函数内部使用 cloud.ai().createModel('cloudbase').generateText()。模型默认配置为 hy3-preview；业务提示词由后端统一持有并传入，避免两处提示词版本漂移。这里仍然需要云函数调用凭据，并非整个系统“无需任何密钥”。

输出结构与容错

提示词要求标题、2—3 句摘要、3—8 条要点与3—6个标签；模型温度设为 0.3，主输入按 12,000 字符上限截断。这些是生成目标，不是效果保证。输出随后进行 JSON 解析、类型规范化和必要截断；非 JSON 输出存在片段兜底，因此不写成“任何情况下都严格输出完整结构”。

针对限流、部分服务端故障和超时，后端按 1、2、4 秒退避，最多进行 4 次调用尝试。不可重试错误或最终失败时，提炼层回退为标题与原文保存，摘要、要点和标签可以为空。长文超出输入窗口时，摘要也不保证覆盖全文。

免费额度应该怎样表达

作者开发期记录显示，项目曾通过微信云开发成长计划赠送的模型额度完成调用，并观察到用量抵扣。这说明平台资源能降低原型验证阶段的模型支出，而不是证明产品永久免费运行。资格、有效期、剩余额度、可用模型及套餐条件，均以参赛时实际控制台规则为准。

> **成本要分开看：赠送额度可抵扣部分模型调用；自建服务器、云环境、带宽、存储、运维和开发投入仍然存在。**

持续运行策略

现有代码可以关闭提炼并保留文字采集能力，降低对单一模型可用性的依赖。正式持续运营还需核实额度到期后的方案、预算与容量。本作品不将赠送额度外推为“保证几十万篇免费提炼”，也不将历史控制台读数当作当前账单。

证据：cloudfunctions/extract/index.js:15、58、104；backend/app/services/llm.py:51、64、140。赠送资源为作者历史记录，本次未登录计费后台复验。

自建文字识别，把数据边界说清楚

开源模型负责识别，自研流程负责组织

项目使用 RapidOCR 与 ONNX Runtime 在自建服务器 CPU 上识别截图，不调用第三方 OCR 接口。模型与开源推理引擎并非参赛者原创；作品的工作在于预处理、任务编排、阅读顺序整理、异常反馈及其与笔记流程的整合。

采用独立 OCR 是成本、部署与输入处理方式的选择，并不意味着多模态大模型不能识字。图片识别后得到的文字仍会发送到腾讯云开发模型进行语义提炼，因此“自建识别”不等于“全流程本地处理”。

处理措施	目的与限制
解码前检查像素量，限制 4,000 万像素	防止压缩文件很小、展开后却消耗大量内存的输入。
识别前将长边缩放至不超过 1600 像素	控制单次推理资源；对细字或极长截图仍可能影响识别，不宣称无损。
OCR 串行执行，按检测框重排并过滤部分噪声	稳定 CPU/内存占用和阅读顺序；复杂排版、表格或特殊语言仍需复核。

真实的数据去向

原始图片：经上传解析、应用暂存、Redis/RQ 任务传递到 worker。代码未将图片作为笔记附件长期保存在业务数据库，但上传框架可能产生临时文件，Redis 是否落盘以及队列参数保留时长取决于部署配置。未完成这些核验前，不能承诺“只在进程内存”“识别后立即全部销毁”或“绝不落盘”。

文字与笔记：抓取正文或识别文本按提炼输入上限发送至腾讯模型；原文和结构化笔记保存在自建业务服务。私有笔记按身份隔离，用户主动分享时才生成公开摘要；删除与账号注销按当前业务实现处理，已保存或打印出去的图片无法收回。

安全与内容责任

抓取侧包含 URL 与网络地址约束；接口包含身份、输入长度及部分频率限制。这些措施不等于完成全面安全审计。分享调用微信内容检测，但检测异常时存在放行路径，不能宣传“所有公开内容均审核通过”。用户也应只导入、处理和分享自己有权使用的资料，并对 AI 结果中的事实与出处复核。

证据：backend/app/services/ocr.py；backend/app/api/routes/ingest.py；backend/app/services/scraper.py、wechat.py。临时文件与队列落盘范围须进一步做部署级确认。

用颜色分流，用文字辨认

我参考的是成熟设计方法，而不是照搬某个产品的功能结论。Bento 式分区强调“一格一义”；Material 的语义色思路提醒我区分功能身份与背景偏好；清晰描边在深浅背景下都能表达组件边界。最终落到“左块右文”：色块提供快速识别，正文承担真实信息。



界面展示 A | 新建入口、链接输入、笔记列表。沿用原稿 v1.1.11 截屏，仅展示当时的交互形态。

从已有动作开始，而不是重新教育用户

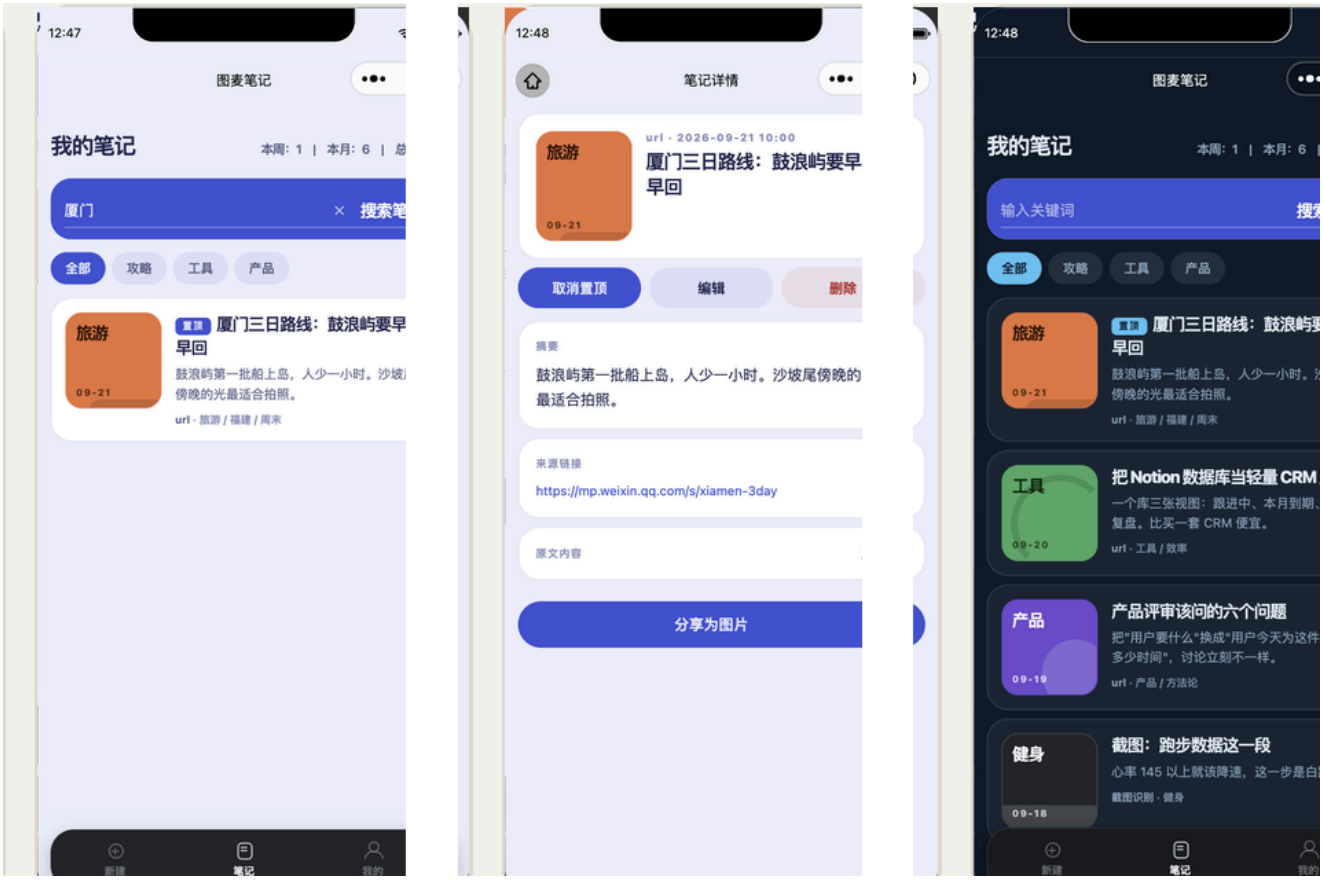
链接、截图与手写各占一个明确入口。展开链接区域即可粘贴和提交，不要求先建立完整知识体系。进入笔记列表后，分类色用于分流，左侧第一标签提示内容主题，标题与摘要帮助判断是否值得打开。

视觉强度服务于内容，而不是盖住内容

高饱和色块只集中在入口与识别位置，阅读区域保持清晰的浅底与描边。颜色、间距、圆角、字体层级集中为设计令牌；更换壁纸时调整语义变量，避免每套外观重新复制一份界面。

配图来自作者原稿的本地界面素材，不作为当前发布版本、实际用户规模或模型识别准确率的证明；其中演示文章仍需遵守来源内容的授权范围。

一张图可以离线看，回流有明确边界



界面展示 B | 关键词搜索、笔记详情、深色外观。原稿 v1.1.11 截屏，分享规则以本文当前代码核对为准。

卡片排版：先测量，再绘制

端侧 Canvas 先测量标题、摘要、要点与标签占用的行数，再计算画布高度并绘制，减少固定高度留下的空白。原生保存相册接口完成导出。视觉生成发生在端侧，不需要额外上传笔记到外部海报生成服务。

公开的是摘要视图，不是私人空间

新生成的分享有效期为 7 天；扫码可查看标题、摘要、标签与关键链接，不直接开放私有笔记全部字段。笔记编辑会同步分享快照，删除笔记会使对应分享失效。知道分享入口的人可访问公开摘要，因此生成分享前应确认内容适合公开。

保存到相册或打印的卡片是当时内容的副本：之后编辑、删除或分享过期，都不会让已经散发的图片自动改变。打印图可以离线阅读；扫码需要网络、可用的小程序入口和仍然有效的分享。这是“数字回流”的边界，而不是永久在线档案承诺。

证据：miniprogram/pages/share/share.js；backend/app/api/routes/shares.py:84—120；backend/app/services/sharing.py。

个人完成整合，AI 辅助而不代替责任

团队形式与成员职责

作品采用个人开发模式。我将产品经理的需求分析与体验设计经验，延伸到小程序、后端和云端能力的整合中，独立统筹产品取舍、交互与视觉设计、工程实现、部署和验证。这种组织方式让需求判断与实现反馈紧密连接，也要求我对作品的稳定性、数据处理与最终交付承担完整责任。

职责	作者承担的工作
产品与体验	从自身信息收集问题出发，定义目标场景、优先级、交互规则与能力边界。
技术整合	搭建原生小程序、自建服务与云函数，接入开源 OCR、模型提炼、存储及分享链路。
验证与维护	设计测试、复核 AI 生成代码、处理异常与资源约束，维护部署和文档口径。

AI 辅助研发的披露

开发过程中使用 Qoder 等 AI 编程辅助工具进行仓库检索、代码草拟、跨文件修改、测试辅助和文案整理。工具与其模型能力属于第三方；作者负责需求决策、代码审查、运行验证及最终提交责任。辅助生成不等于自动正确，也不把工具产出的通用实现包装成独创算法。

如何验证，而不是只说“已经跑通”

后端测试覆盖采集、模型异常、用户隔离、额度与邀请、注销、分享快照、输入约束等；客户端使用小程序开发者工具与 miniprogram-automator 做交互检查。测试替身可以验证协议和错误分支，但不能证明真实 OCR 质量、模型响应时间或生产服务可用性。

从 2026 年 9 月 17 日的首个可运行闭环，到 9 月 22 日的界面与分享流程完善，首个开发阶段历时六天。AI 辅助扩大了个人可完成的工作范围，但没有替代人的判断：哪些能力值得接入、哪些承诺必须收回、哪一版界面需要推翻，仍由开发者决定。后续持续完善识别质量、错误反馈与上线验收，而不是把短期原型等同于长期运营已经完成。

建议评审体验路径

用一条公开文章链接生成笔记并核对来源；导入一张清晰截图检查文字与要点；手写一条记录并分类、编辑、搜索；生成卡片并查看公开摘要。再试无正文链接和空白图片，观察是否如实提示，而不是留下空成品。建议使用演示账号与已授权材料，不提交私人内容。

本地版本标识：miniprogram/pages/about/about.js:6；验证材料位置：backend/tests/、backend/probes/、docs/工具/。测试文件的存在不作为全部通过证明。

07 · 非团队开发成果引用

明确来源，也明确自己的贡献

本作品引用以下平台与开源成果。其代码、模型和服务能力归各原作者或权利人所有，参赛者不声称自行开发或训练这些基础能力。版本以项目依赖声明及本地安装元数据为依据，不据此推断所有生产环境完全一致。

成果 / 来源	用途与引用范围	本次可核对的许可或条件
腾讯微信小程序、微信云开发与混元模型	客户端运行、登录、小程序码、内容检测、云函数及文本提炼。	平台服务，遵循相应服务协议、调用限制及套餐规则；不是本项目自研模型。
RapidAI / RapidOCR 3.9.2；PP-OCR 系识别模型	截图文字识别，使用现成模型与推理封装。	RapidOCR 软件元数据为 Apache-2.0；模型权重须按实际文件另核来源与许可。
Microsoft / ONNX Runtime	CPU 模型推理；项目约束 >=1.23.2,<2。	本地安装包为 MIT；部署环境解析版本可能不同。
trafilatura 1.12.2；BeautifulSoup4 4.12.3；lxml	网页正文抽取、HTML 解析与清理。	分别为 Apache-2.0、MIT；lxml 本地元数据为 BSD-3-Clause。
OpenCV-Python 5.0.0.93；Pillow 10.4.0；NumPy	图像预处理与数组运算。	OpenCV 包为 Apache-2.0；Pillow 为 HPND；NumPy 包含多项许可声明。
FastAPI / Uvicorn；SQLAlchemy / Alembic	API 服务、数据库访问与迁移。	FastAPI、SQLAlchemy、Alembic 为 MIT；Uvicorn 为 BSD-3-Clause。
Redis 服务；redis-py 8.1.0；RQ 2.12.0	异步队列与任务状态。	Python 客户端为 MIT、RQ 为 BSD-2-Clause；Redis 服务端许可需按部署版本另核。
Qoder；pytest；miniprogram-automator	AI 辅助开发、单元测试与小程序自动化。	按各工具条款使用；本地 pytest 元数据为 MIT；工具能力不作为本作品原创。

来源定位：backend/requirements.txt、cloudfunctions/extract/package.json、各安装包 METADATA / LICENSE。许可名称是识别引用来源的依据，不代替完整开源合规检查，也不自动覆盖模型、字体、图片及捆绑二进制。

第三方引用与技术证据索引

其余直接依赖与用途

Pydantic 2.9.2、pydantic-settings 2.6.1（MIT）：输入模型与配置；python-multipart 0.0.12（Apache-2.0）：上传解析；HTTPX 0.27.2（BSD-3-Clause）：HTTP 调用；lxml_html_clean 0.4.2（BSD-3-Clause）：HTML 清理；PyJWT 2.9.0、slowapi 0.1.9（MIT）：令牌与限流；qrcode 7.4.2（三条款 BSD 主许可并包含其他来源声明）：二维码图像能力；psycopg2-binary 2.9.10（LGPL 附例外）：PostgreSQL 驱动。wx-server-sdk 的声明版本为 >=4.0.1，许可及平台条件以实际 SDK 分发内容为准。

上游来源索引

RapidOCR：rapidai.github.io/RapidOCRDocs；ONNX Runtime：onnxruntime.ai；trafilatura：github.com/adbar/trafilatura；BeautifulSoup：www.crummy.com/software/BeautifulSoup/bs4/；FastAPI：fastapi.tiangolo.com；RQ：python-rq.org；OpenCV-Python：github.com/opencv/opencv-python；Pillow：python-pillow.org。以上地址来自本地包的项目元数据，用于查明来源，并非本次在线审阅记录。

素材、设计参考与文档工具

界面展示取自作者原稿的本地截屏，仅作为作品界面演示；示例文章内容、壁纸、字体和其他视觉素材仍归各自权利人所有，使用与参赛展示应具备相应授权。Bento、Material 等只作为交互设计参考，不主张对这些通用范式拥有原创权。

本版说明经 AI 辅助整理和技术核对，使用 python-docx、Pillow 与浏览器排版生成可编辑文档及 PDF；这些是第三方文档工具，不属于作品的运行时创新。模型权重的具体文件、Redis 服务端版本及素材授权材料仍需由提交者核实，本文不声明已完成全部许可审计。

关键实现的核对入口

说明内容	项目内证据
采集与保存	miniprogram/pages/create/create.js；backend/app/tasks/ingest_tasks.py
AI / OCR	backend/app/services/llm.py、ocr.py；cloudfunctions/extract/index.js
分享与身份	backend/app/api/routes/shares.py；backend/app/services/sharing.py；backend/app/core/auth.py
依赖与测试	backend/requirements.txt；backend/tests/；cloudfunctions/extract/package.json

> **图麦笔记的价值，不是替用户保存更多，而是让已经发现的好内容，更容易成为下一次真正用得上的积累。**